Understand Array Index Range and Zero-Based Indexing

---

1. What is Zero-Based Indexing?

In Java, **array indexing starts from 0**, not 1.

Example:

```
int[] numbers = {10, 20, 30, 40};
```

Index	0	1	2	3
Value	10	20	30	40

First element:

```
numbers[0]
```

Last element:

```
numbers[3]
```

---

2. Valid Index Range

For an array of size **n**:

```
Valid Index = 0 to (n - 1)
```

Example:

```
int[] arr = new int[5];
```

Indexes:

```
0 1 2 3 4
```

Not valid:

```
5
-1
```

---

3. Accessing Elements

```
int[] arr = {5, 10, 15};

System.out.println(arr[0]);   // 5
System.out.println(arr[2]);   // 15
```

---

4. What Happens for an Invalid Index?

```
int[] arr = {10, 20, 30};

System.out.println(arr[3]);
```

Output:

```
ArrayIndexOutOfBoundsException
```

The same happens for negative indexes:

```
arr[-1];
```

---

5. Interview Insight: Why Does Java Use Zero-Based Indexing?

Arrays are stored in **contiguous memory**.

The address of an element is calculated as:

```
Base Address + (Index × Size of Element)
```

The **first element is at an offset of 0**, so Java starts indexing from 0. This makes address calculation simple and efficient.

> **Interviewer's expectation:** You don't need to memorize the formula, but you should know that **zero-based indexing is chosen for efficient memory access**, not arbitrarily.

Common Mistakes

❌ Using 1 as the first index.

```
arr[1];
```

This accesses the **second** element.

❌ Using <= instead of < in loops.

```
for (int i = 0; i <= arr.length; i++)
```

This causes:

```
ArrayIndexOutOfBoundsException
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Tricky Interview Questions

Q1. What is the valid index range of an array of size 10?

Answer:

```
0 to 9
```

Q2. What is the last index of an array?

Answer:

```
arr.length - 1
```

Not arr.length.

Q3. Why doesn't Java start array indexing from 1?

Answer:

Because **the first element is at a memory offset of 0**, making array access faster and simpler.

Output-Based Questions

Question 1

```
int[] arr = {10, 20, 30};

System.out.println(arr[0]);
```

Output

```
10
```

Question 2

```
int[] arr = {10, 20, 30};

System.out.println(arr[arr.length - 1]);
```

Output

```
30
```

Question 3

```
int[] arr = {10, 20, 30};

System.out.println(arr[3]);
```

Output

```
ArrayIndexOutOfBoundsException
```

Cheat Sheet

Concept	Value
First index	0
Last index	length - 1
Valid range	0 to length - 1
Invalid index	< 0 or >= length
Common exception	ArrayIndexOutOfBoundsException

> **Interview Tip:** A very common bug is writing:
>
> ```
> for (int i = 0; i <= arr.length; i++)
> ```
>
> Always ask yourself: **"Is i allowed to become arr.length?"**
> The answer is **no**, because the last valid index is **arr.length - 1**. This simple reasoning helps avoid one of the most common array errors in Java.